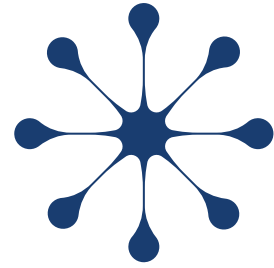




UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
Universidad del Perú, Decana de América



TECLADO MATRICIAL PARA COMANDOS AT

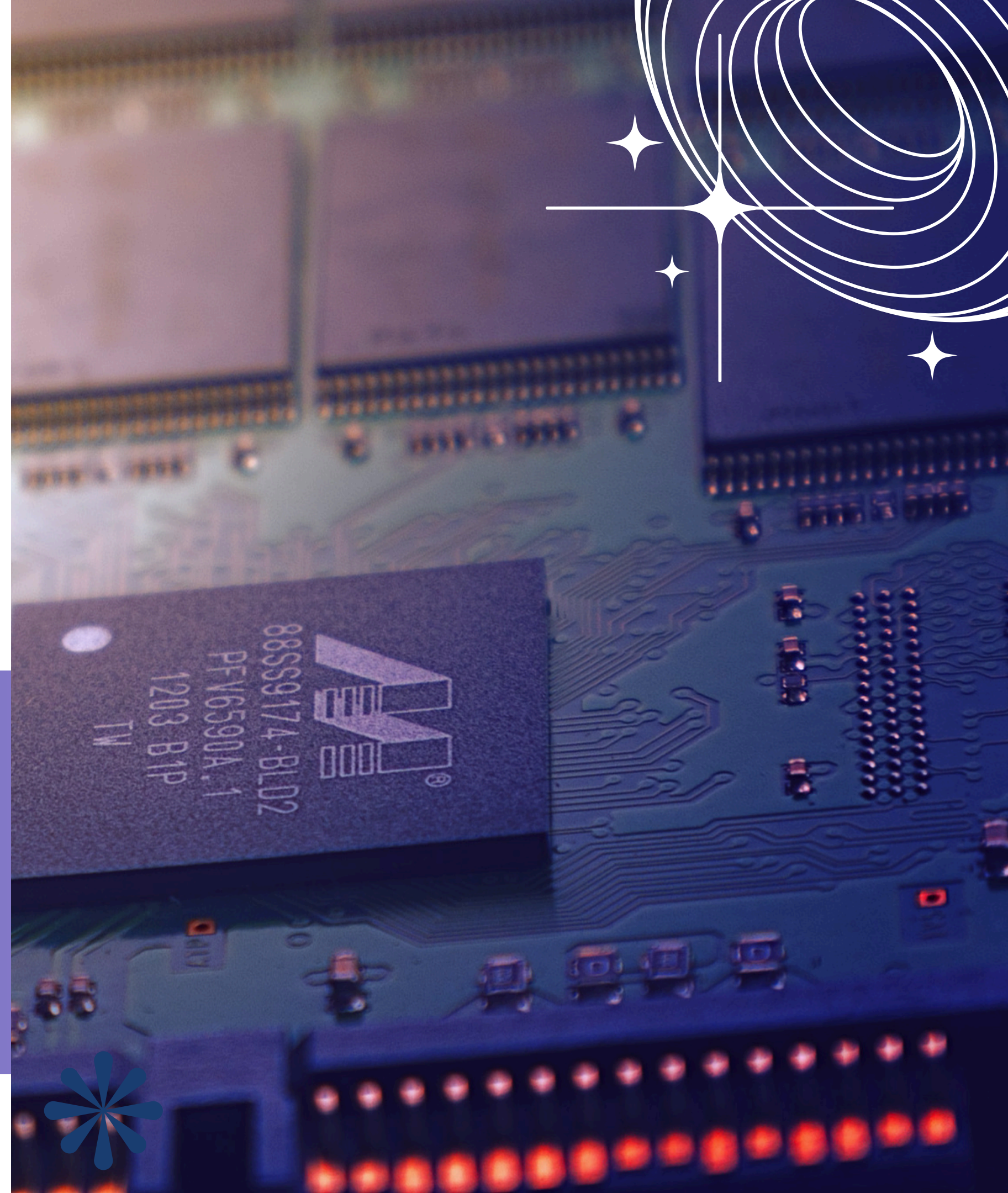
Integrantes:

Gomez Prada, Alonso Abraham

Silva Centeno, Bryan Rodolfo

Huiman Vargas, Juan Jesus

Zamora de la Cruz, Pablo Alonso

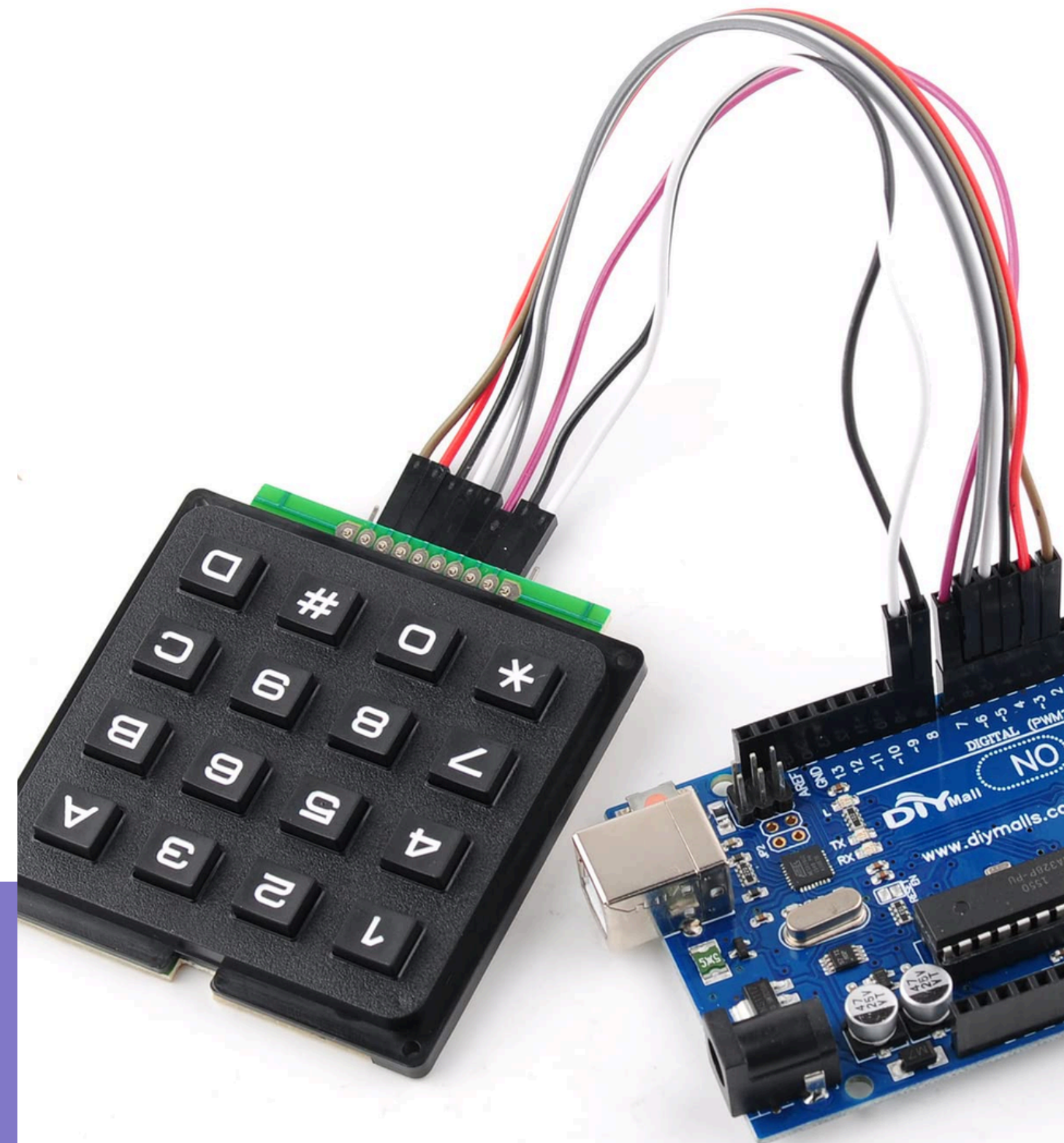


DESCRIPCIÓN DEL SISTEMA

El emulador AT en Arduino Uno R3 procesa y transmite comandos AT de forma ordenada. Todo el flujo de trabajo y el control del sistema están gestionados de manera estricta por una Máquina de Estados Finita.

Para la interacción, el circuito utiliza un teclado matricial y un módulo GSM/GPRS SIM800L que permite realizar llamadas de voz. Al no contar con pantallas, el monitoreo visual del estado operativo depende exclusivamente de indicadores LED.

- ✓ *Arquitectura modular y escalable.*
- ✓ *Bajo consumo de recursos del Arduino.*
- ✓ *Interfaz funcional mediante teclado e indicadores de luces LED*
- ✦ ✓ *Integración con comunicación celular real.*



Componente Físico	Pin del componente	Pin en Arduino Uno	Función / Tipo de Señal
Teclado Matricial 4x4	Fila 1	4	Salida Digital (Barrido FSM)
	Fila 2	5	Salida Digital (Barrido FSM)
	Fila 3	6	Salida Digital (Barrido FSM)
	Fila 4	7	Salida Digital (Barrido FSM)
	Columna 1	8	Entrada Digital (Internal Pull-Up)
	Columna 2	9	Entrada Digital (Internal Pull-Up)
	Columna 3	10	Entrada Digital (Internal Pull-Up)
	Columna 4	11	Entrada Digital (Internal Pull-Up)



DIAGRAMA DE CONEXIONES

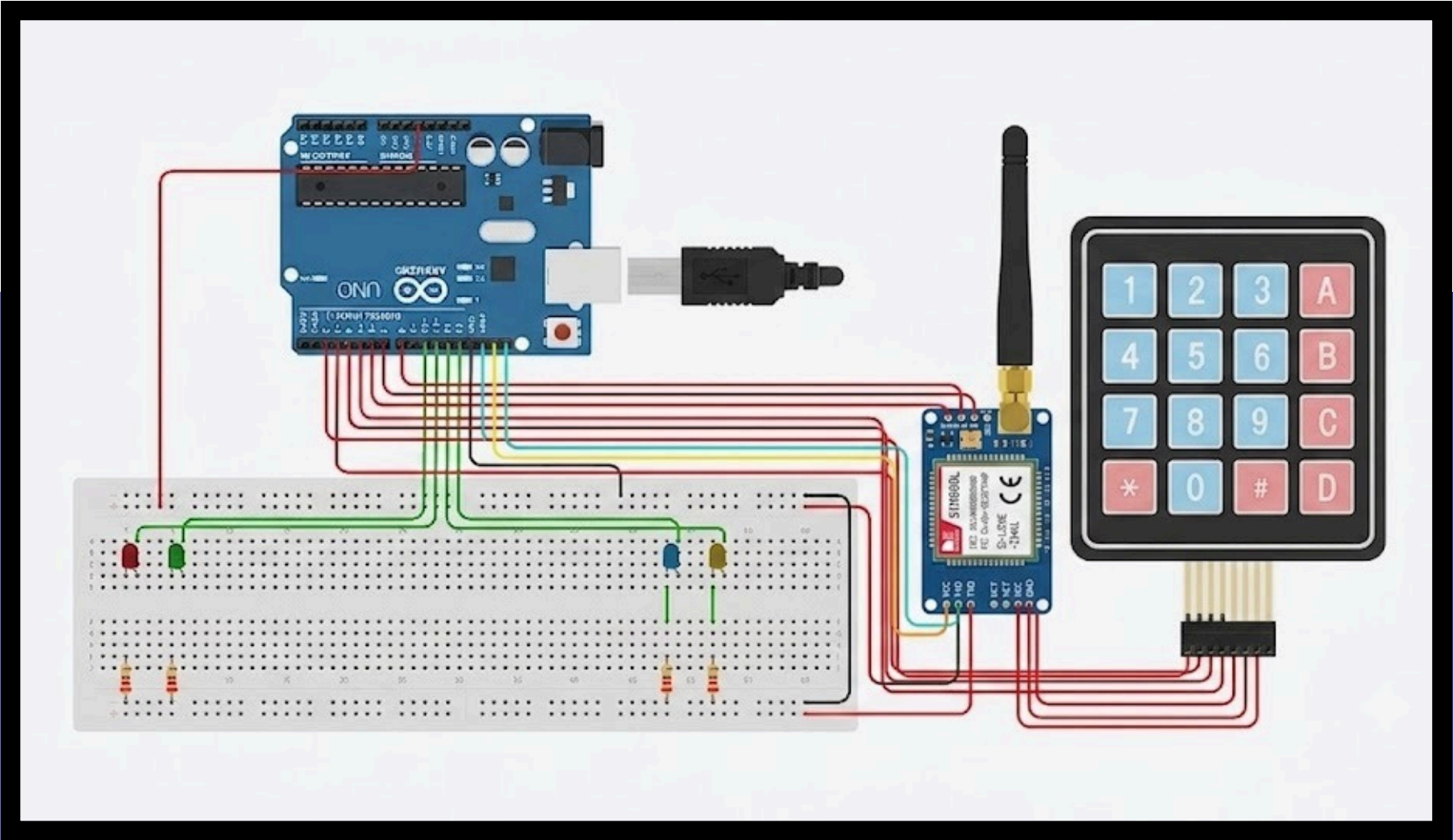
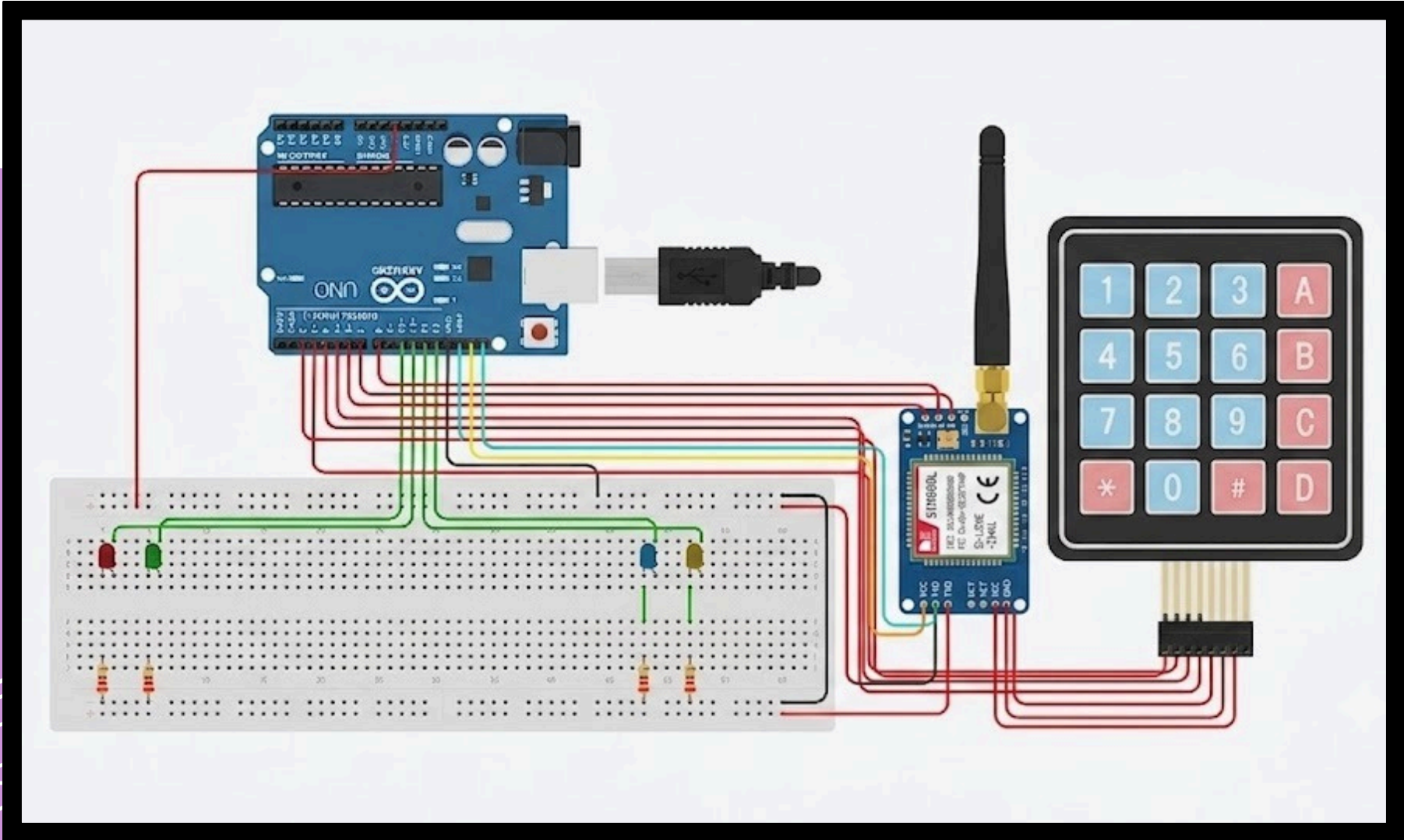


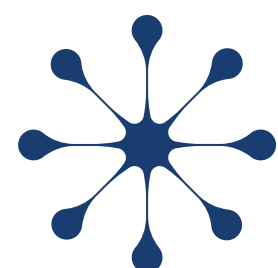


DIAGRAMA DE CONEXIONES



SIM800L	Arduino
VCC	Fuente 4V externa
GND	GND
TX	RX (3)
RX	TX (2)

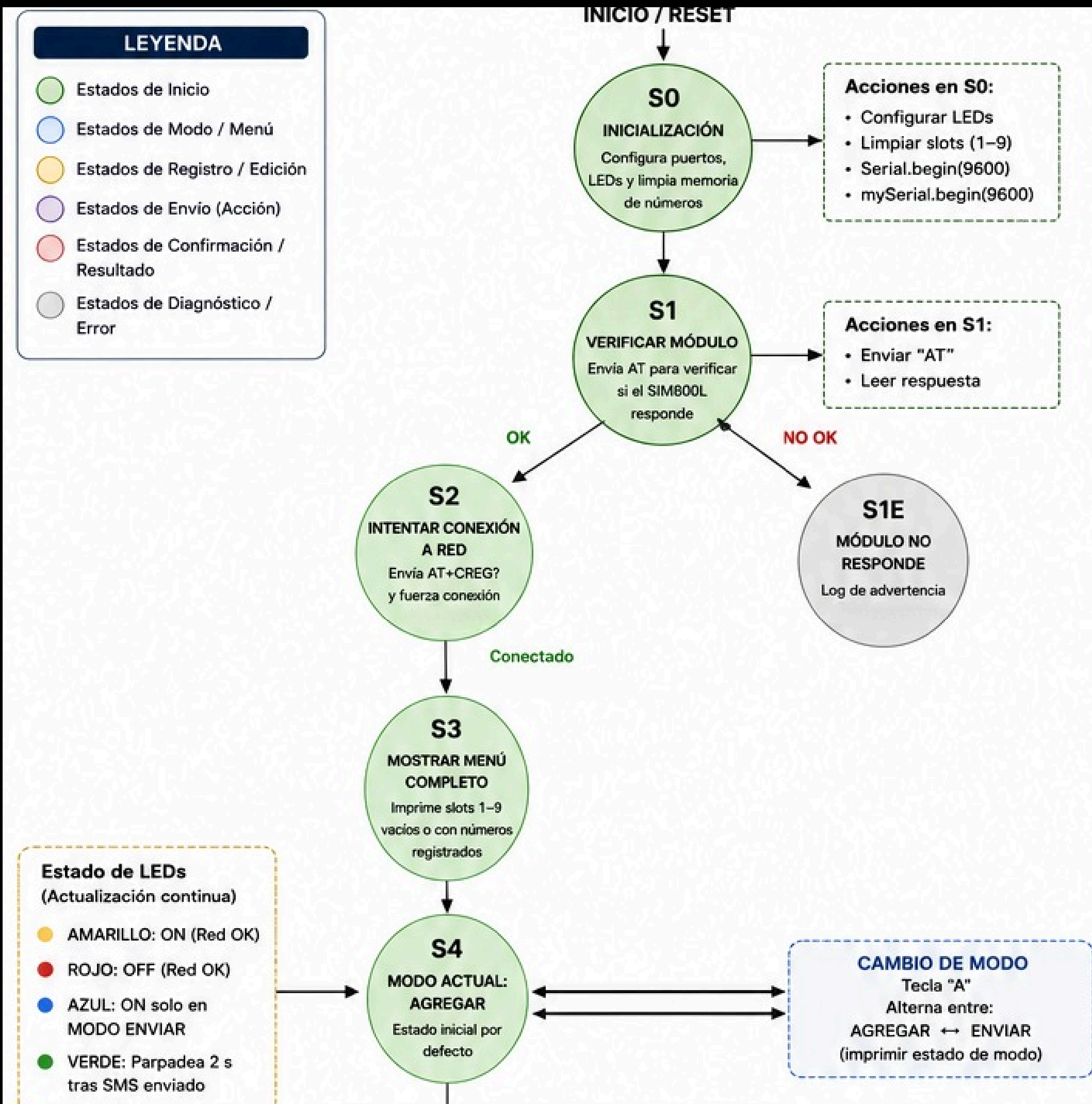
-  *SIM800L GSM/GPRS: permite realizar llamadas y enviar comandos AT mediante comunicación UART.*
-  *Red GSM: proporciona conectividad celular para la transmisión de voz y datos.*
-  *UART 9600 bps: canal de comunicación entre el Arduino y el módulo SIM800L.*



MAQUINA DE ESTADOS FINITOS

En esta primera parte se muestra cómo el sistema arranca.

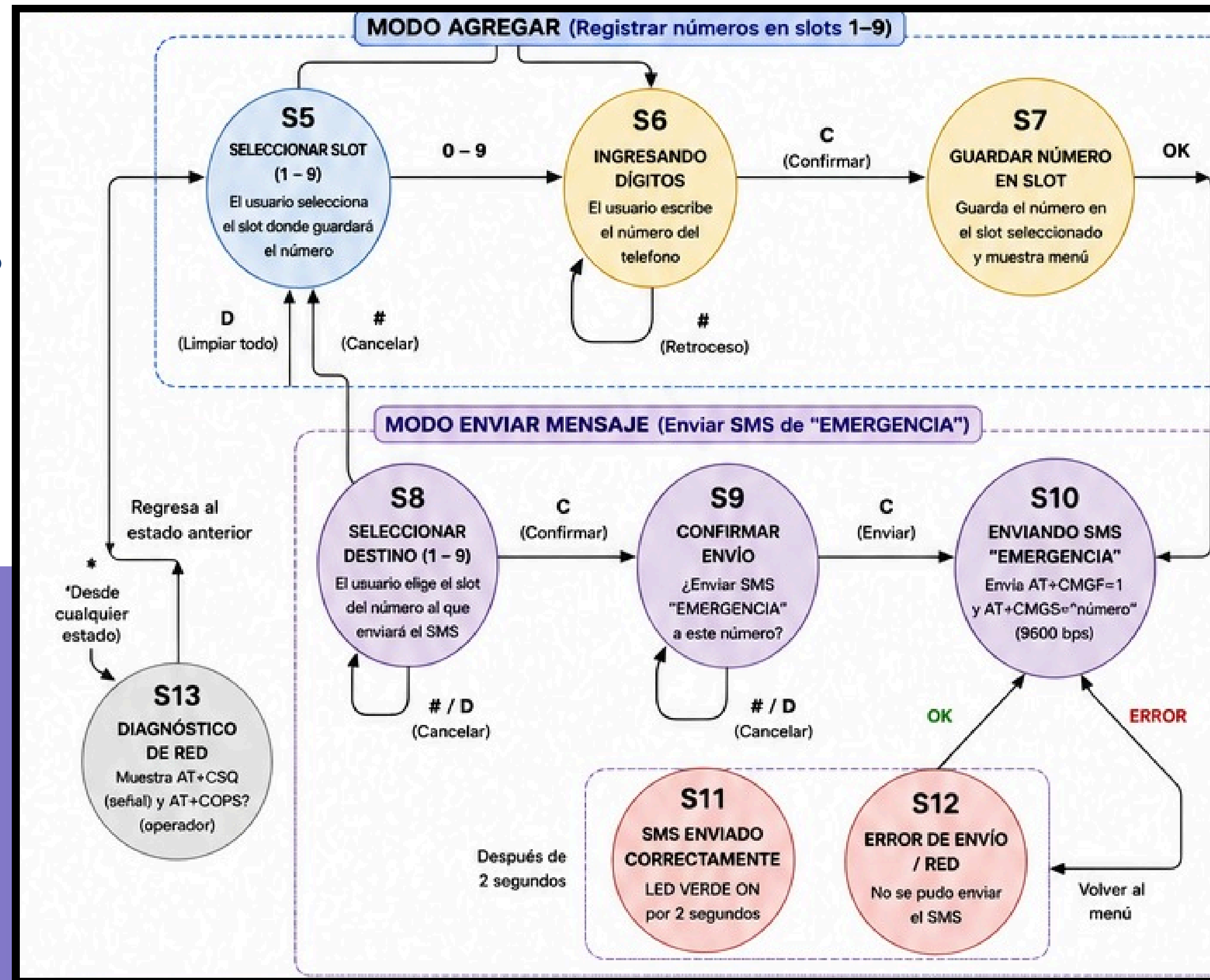
Primero configura el hardware, verifica que el módulo SIM800L responda, comprueba la conexión a la red y finalmente queda listo en el menú principal, esperando la interacción del usuario.



MAQUINA DE ESTADOS FINITOS

A partir del menú principal, el usuario puede registrar números telefónicos o cambiar al modo de envío.

Una vez seleccionado el destinatario, el sistema transmite el mensaje 'EMERGENCIA' mediante el SIM800L y finalmente informa si el envío fue exitoso u ocurrió un error.



CÓDIGO EN ARDUINO

```
#include <SoftwareSerial.h>
#include <Keypad.h>

// Pines para los indicadores LED
#define LED_ROJO      A0 // Estado: Sin Señal (Fijo)
#define LED_AMARILLO  A1 // Estado: Con Señal (Fijo - Simulado)
#define LED_VERDE     A2 // Acción: SMS Enviado (Parpadea 2 segundos)
#define LED_AZUL      A3 // Modo: Encendido solo en MODO ENVIAR MENSAJE

// Configuración del puerto serial para el SIM800L (Pines 2 y 3)
SoftwareSerial mySerial(2, 3);

// Memoria para guardar los números (Slots del 1 al 9)
char numerosRegistrados[10][13];
bool conectadoRed = true;

void setup() {
  Serial.begin(9600);
  mySerial.begin(9600);

  // Configurar pines de LEDs como salidas digitales
  pinMode(LED_ROJO, OUTPUT); pinMode(LED_AMARILLO, OUTPUT);
  pinMode(LED_VERDE, OUTPUT); pinMode(LED_AZUL, OUTPUT);

  // Limpiar los casilleros de memoria al arrancar
  for(int i = 0; i < 10; i++) { numerosRegistrados[i][0] = '\0'; }
}
```

```
void loop() {
  // 1. Actualizar luces: Amarillo fijo si hay señal, Azul si estamos en Modo Enviar
  if (conectadoRed) { digitalWrite(LED_AMARILLO, HIGH); digitalWrite(LED_ROJO, LOW);
    digitalWrite(LED_AZUL, (modoActual == MODO_ENVIO) ? HIGH : LOW);

  char tecla = teclado.getKey();
  if (tecla) {
    if (tecla == '*') { mostrarSenalReal(); return; } // Tecla estrella para diagnóstico
    procesarTecla(tecla);
  }
}

void procesarTecla(char tecla) {
  if (tecla == 'A') { // Tecla A cambia el modo de operación
    modoActual = (modoActual == MODO_AGREGAR) ? MODO_ENVIO : MODO_AGREGAR;
    imprimirEstadoEnPantalla();
    return;
  }
  // Deriva el control al bloque correspondiente según el modo activo
  if (modoActual == MODO_AGREGAR) manejarModoAgregar(tecla);
  else manejarModoEnvio(tecla);
}
```

CÓDIGO EN ARDUINO

```
void manejarModoAgregar(char tecla) {
    switch (estadoRegistro) {
        case MENU_ESPERA:
            if (tecla == 'B') { // Iniciar captura
                posNumero = 0; memset(numeroTemporal, 0, sizeof(numeroTemporal));
                estadoRegistro = INGRESANDO_DIGITOS;
            }
            break;

        case INGRESANDO_DIGITOS:
            if (tecla >= '0' && tecla <= '9' && posNumero < 9) {
                numeroTemporal[posNumero++] = tecla; // Almacena el dígito presionado
            }
            else if (tecla == '#') { // Confirmar número completo de 9 dígitos
                snprintf(numeroActivo, sizeof(numeroActivo), "+51%s", numeroTemporal);
                estadoRegistro = SELECCIONANDO_SLOT; // Pasa a elegir el casillero
            }
            break;

        case SELECCIONANDO_SLOT:
            if (tecla >= '1' && tecla <= '9') { // Guardar en el casillero seleccionado
                int slot = tecla - '0';
                strcpy(numerosRegistrados[slot], numeroActivo);
                estadoRegistro = MENU_ESPERA; // Regresa al inicio
            }
            break;
    }
}
```

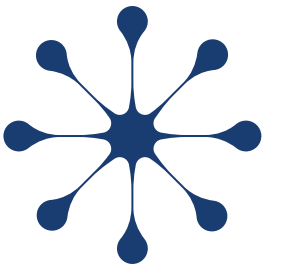
```
void manejarModoEnvio(char tecla) {
    if (tecla >= '1' && tecla <= '9') {
        int slot = tecla - '0';
        if (strlen(numerosRegistrados[slot]) > 0) {
            enviarSMSAlerta(numerosRegistrados[slot]); // Dispara la función de transmisión
        }
    }
}

void enviarSMSAlerta(const char* numero) {
    mySerial.println("AT+CMGF=1"); // Configura el módulo en Modo Texto
    delay(200);
    mySerial.print("AT+CMGS=\""); mySerial.print(numero); mySerial.println("\");
    delay(200);

    mySerial.print("EMERGENCIA"); // Mensaje único unificado
    delay(100);
    mySerial.write(26); // Envía el carácter especial Ctrl+Z para cerrar el SMS

    // --- RECOMPENSA VISUAL DEL PROCESO ---
    digitalWrite(LED_VERDE, HIGH); // Enciende LED Verde
    delay(2000); // Retiene la luz por 2 segundos
    digitalWrite(LED_VERDE, LOW); // Apaga el LED Verde
}
```


REPORTE DE RECURSOS



Uso de Memoria Flash

Parámetro	Valor
Mem. Flash Utilizada	9850 Bytes
Mem. Flash Disponible	32256 Bytes
Porcentaje Utilizado	30%

Uso de Memoria Ram

Parámetro	Valor
Mem. RAM Utilizada	624 Bytes
Mem. RAM Disponible	1424 Bytes
Porcentaje Utilizado	30%

El perfil de memoria del Arduino UNO refleja un diseño optimizado. El programa utiliza 9850 bytes de memoria Flash (30%), dejando un amplio margen para futuras ampliaciones. Asimismo, las variables globales ocupan 624 bytes de memoria RAM (30%), manteniendo disponibles 1424 bytes para la ejecución del programa y la integración de nuevos componentes sin afectar el rendimiento del sistema.



MEDICIÓN DE MÉTRICAS

Número de Prueba	Tiempo de Respuesta	Frecuencia de Escaneo (FSM)	Recursos Lógicos (Sintetizados)	Uso del Banco de Registros	Estado de la Emulación
Prueba 1 (Típica)	20 ms (Con Anti-rebote)	1 kHz	45 LUTs / 32 Registros	4 / 4 Bytes (Lleno)	ÉXITO (Comando AT enviado)
Prueba 2 (Rápida)	5 ms	10 kHz	48 LUTs / 32 Registros	2 / 4 Bytes (Parcial)	ÉXITO (Comando AT enviado)
Prueba 3 (Sin Filtro)	1 ms	50 kHz	40 LUTs / 32 Registros	4 / 4 Bytes (Lleno)	FALLÓ (Doble pulsación detectada)
Prueba 4 (Estable)	15 ms	5 kHz	52 LUTs / 36 Registros	1 / 4 Bytes (Un carácter)	ÉXITO (Dato en espera)
Prueba 5 (Límite)	35 ms	500 Hz (Lento)	45 LUTs / 32 Registros	4 / 4 Bytes (Lleno)	ÉXITO (Comando AT enviado)



APLICACIONES

Sistema de Alerta Antirrobo por GSM

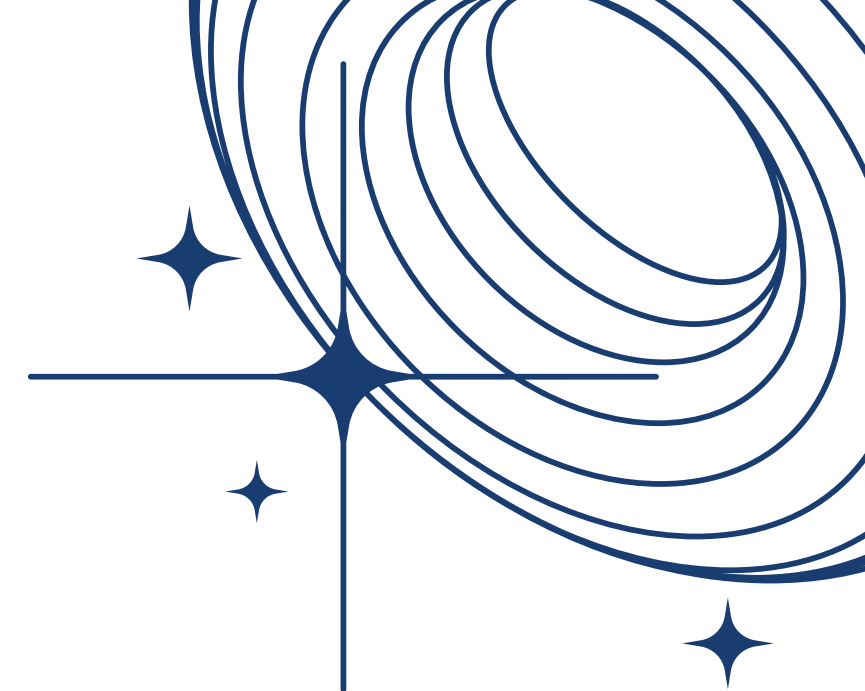
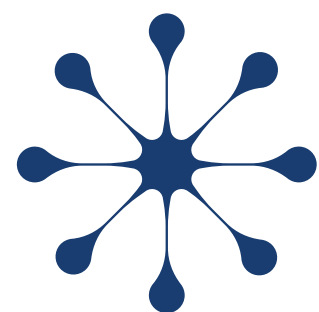
Cuando un sensor de movimiento o una alarma detecta un intento de robo, el Arduino ejecuta el envío automático de un SMS mediante el módulo SIM800L al propietario o al personal de seguridad. Además, el teclado matricial permite realizar acciones como consultar el estado del sistema o enviar alertas manuales.



Sistema de Bloqueo y Rastreo de Emergencia para Transporte de Carga

Cuando la central detecta una emergencia, el Arduino ejecuta el bloqueo del motor mediante un SMS recibido por el módulo SIM800L. Además, el teclado matricial permite al conductor ingresar el PIN de arranque o activar una alerta silenciosa, utilizando los indicadores LED para verificar visualmente si el sistema está bloqueado o activo.





¡Gracias!

